Homework Assignment 04 & 05

Part 1:

Objective:

- To implement insertion and deletion operations in a Binary Search Tree and AVL tree using a doubly linked list as well as visualize the process of balancing with rotations and deleting nodes that are in a tree.

Approach and methodology

- My approach was to create a BST and AVL Tree using a DoublyNode class which includes references to its left and right children. I began by generating random numbers in a range of 10 to 99 and made sure each value was unique before storing them into the ArrayList. These values were then inserted to the BST and AVL tree with each tree having its own ArrayList of numbers. For the BST the insertion will continue until the tree reaches a height of 5 and once this height is reached the BST is printed. This BST will have a loop that deletes a node and will continue to print after every deletion until the BST is empty. For the AVL three 35 unique values were inserted. After every insertion the tree was printed and the balance was checked to ensure that the AVL tree remained balanced. After the creation of the tree the root was repeatedly deleted and then the tree was reprinted.

Analysis of Trial

- **Insertion Analysis**
    - When inserting a node to a BST there is no rebalancing therefore it is simple to add a node. This results in faster insertions for datasets that are smaller. A BST could become a bad option because the tree can have a time complexity of O(n) in the case where the tree has a bias to one side. For the AVL tree despite the insertions being more complex due to the balances and rotations, the tree structure is the best it can be. Due to this the AVL tree has an O(log n) height which is very beneficial for larger data sets.
- Deletion Analysis
    - The deletion of a BST can be bad if the root is removed compared to a leave due to there being a need to search for a value to replace the root. However there is no balancing required which may result in a tree that is very unbalanced. In a AVL tree the tree rebalances itself and maintains its ideal height after the deletion of each root. It does require more work to

maintain this height however this allows for future insertions and deletions to still be efficient.

- **Time Complexity Analysis**
  - Insertion BST
    - The average time complexity of this method would be O(log n) with the worst case being O(n) if the tree ended up being very unbalanced or even a linear tree.
  - Insertion AVL
    - Due to the balance of the tree the time complexity will be O(log n).
  - Deletion BST
    - The average time complexity of deletion would be O(log n) and the worst case would be O(n).
  - Deletion AVL
    - Due to the tree being balanced the time complexity will be O(log n).
  - Searching BST
    - To search in a BST the average time complexity is O(log n) and the worst is O(n) in the case where the tree is very unbalanced or even a linear tree.
  - Searching AVL
    - The average time complexity of searching a binary tree is O(log n).

Conclusion

- The BST implementation showcases how a basic tree structure works and how the deletion of a root can affect the overall height and shape of a tree. In datasets where the maximum height is 5 the BST is much more efficient than what it would be like with larger sets due to it not having a balancing component. The AVL tree is a little slower in the more basic operations with small data sets but as the number of nodes grows it becomes a better option compared to the BST due to its balancing property which maintains a height and structure that is best worked with. The rotations are what give the AVL tree the upperhand in larger data sets on BST. Overall, the AVL Tree is the better option when working with data sets that are on the larger size due to the balancing optimizing the functionality. This doesn't mean that the BTS is useless, it still is a good option for smaller data.

Part 2:

Objective

- This assignment's objective is to implement insertion, search and mixed operations to Binary Search Trees and AVL trees to compare the performance of the two trees. The overall goal of this assignment was to determine how the self-balancing property impacts the AVL tree in time complexity and runtime and to see which of the two is more efficient based on the test case

Approach and methodology

- For task 1 my approach was to create a BST and AVL tree using random integers which were the same for both trees and they were inserted into each tree. I started with generating a set of random integers and stored them in a list which was the ArrayList that was used for both trees. When the insertion started for both trees the process began with two empty trees, one BST and one AVL. In order to record the time it took to implement everything I used System.nanoTime() to calculate the time it took for everything to be inserted to its respective tree. Due to the AVL tree needing to be a balanced tree there were additional steps such as balance factors and rotations whenever needed during the insertion resulting in the time to insert something being O(log n). On the other hand, the BST is a much simpler tree when it comes to insertions as there is no balancing required and the worst case to insert something would be O(n).
- For task 2 my approach was to evaluate the search performance with the use of random integers to search for in both trees that were created in task 1. These random values will result in successful and unsuccessful searches. In order to evaluate the efficiency, the performance was timed with the help of System.nanoTime() and the nanoseconds were calculated. The search values consisted of 500 known integers and 500 integers that were not present in the trees.
- For task 3 my approach was to implement mixed operations that had different ratios of insertions and searches such as 25/75, 50/50, and 75/25. My approach was to simulate situations where both insertions and searches were being used in real world applications. When doing this task I made an operation array that randomly performed an insert and search operation based on the ratio. The reason an array was used for this was to ensure that moth trees had a fair and equal execution without bias. This approach showcases the performance of both trees depending on what functions need to be performed with each tree having an advantage in their own respective area whether it is inserting something or searching.

Sample trial images

```
Test with n = 1000
------------------------
BST insertion time 2330500.0 ns
AVL insertion time 4847708.0 ns
BST search time 1821166.0 ns
AVL search time 1112125.0 ns

BST successful search time: 1135333.0 ns
AVL successful search time: 900000.0 ns
BST unsuccessful search time: 240917.0 ns
AVL unsuccessful search time: 246667.0 ns

Mixed Operation (Insert 25%, Search 75%):
BST mixed op time: 772500.0 ns
AVL mixed op time: 830292.0 ns

Mixed Operation (Insert 50%, Search 50%):
BST mixed op time: 202750.0 ns
AVL mixed op time: 207791.0 ns

Mixed Operation (Insert 75%, Search 25%):
BST mixed op time: 814417.0 ns
AVL mixed op time: 488084.0 ns


Test with n = 10000
------------------------
BST insertion time 1960167.0 ns
AVL insertion time 1537792.0 ns
BST search time 242250.0 ns
AVL search time 410042.0 ns
```

```
BST successful search time: 118750.0 ns
AVL successful search time: 354917.0 ns
BST unsuccessful search time: 127959.0 ns
AVL unsuccessful search time: 247625.0 ns

Mixed Operation (Insert 25%, Search 75%):
BST mixed op time: 659500.0 ns
AVL mixed op time: 618166.0 ns

Mixed Operation (Insert 50%, Search 50%):
BST mixed op time: 330708.0 ns
AVL mixed op time: 456709.0 ns

Mixed Operation (Insert 75%, Search 25%):
BST mixed op time: 264875.0 ns
AVL mixed op time: 290916.0 ns
```

```
Test with n = 100000
-----------------------
BST insertion time 1.4461209E7 ns
AVL insertion time 1.8486083E7 ns
BST search time 457333.0 ns
AVL search time 399375.0 ns

BST successful search time: 71333.0 ns
AVL successful search time: 103750.0 ns
BST unsuccessful search time: 268625.0 ns
AVL unsuccessful search time: 233917.0 ns

Mixed Operation (Insert 25%, Search 75%):
BST mixed op time: 1114750.0 ns
AVL mixed op time: 2173833.0 ns

Mixed Operation (Insert 50%, Search 50%):
BST mixed op time: 763625.0 ns
AVL mixed op time: 1153208.0 ns

Mixed Operation (Insert 75%, Search 25%):
BST mixed op time: 1500375.0 ns
AVL mixed op time: 1305916.0 ns
```

Analysis of sample trial

- **Insertion Analysis**

    - At n = 1,000 the insertion time of a BST was significantly lower in this case. This reflects the effect and cost that the balancing and rotations of an AVL tree have on it. When inserting for a tree that was n = 10,000 the AVL insertion was faster in this case which may have happened due to better balancing behavior. At n =100,000 the AVL insertion time once again exceeded the BST time. This shows that as the dataset grows in size, the AVL becomes the more favorable option. Overall, AVL insertion is O(log n) and BST can occasionally appear faster but is a less reliable choice as n increases in size.
- **Search Analysis**
    - In all three cases the AVL tree performed better or similarly compared to the BST. At n = 1,000, AVL outperformed the BST as well as in the other two trials (10,000 and 100,000). This is because the self-balancing of the AVL tree results in a height of O(log n) resulting in more efficient and consistent searches.
- **Successful and Unsuccessful searches**
    - For successful searches BST was sometimes faster than the AVL tree. Overall though, the AVL tended to perform better than the BST in unsuccessful searches likely due to the BST being unbalanced and having a larger cost if a search compared to the AVL's height being balanced.
- **Mixed Operations Analysis**
    - For 25% insert / 75% search the AVL tree outperformed the BST at n = 10,000 however it was slower in the other two trials.
    - For 50% insert / 50% search the performance was similar in the three trials but the AVL tree began to fall behind in efficiency as there were more nodes being inserted resulting in larger balancing costs.
    - For 75% insert / 25 %search the AVL tree was faster than the BST which showcases that the tree's O(log n) tree height becomes helpful in scenarios where a dataset is large.
- Time Complexity Analysis
    - Insertion BST
        - The average case to insert a value is O(log n) and the worst case is O(n) which occurs when the tree is unbalanced.
    - Insertion AVL
        - Due to the height balancing with the use of rotation this time complexity will be O(log n),
    - Searching BST
        - The average case for searching would be O(log n) with the worst case being O(n) when a tree is unbalanced.
    - Searching AVL

        - Due to the height balancing with the use of rotation this time complexity for searching will be O(log n),
    - Mixed Operations
        - BST
            - This will have an average of O(n log n) time complexity, however it can also be $O(n^2)$ in the worst case.
        - AVL
            - This will have a time complexity of O(n log n).

Conclusion

- The BST looks to offer faster insertions for datasets that are smaller due to it being a simple tree with not as much going on as the AVL tree. As the datasets become larger the AVL trees are much more consistent with their performance and their balancing function makes it to where they are always efficient. Overall, AVL trees are better for larger applications and the BST may be more efficient in smaller datasets.